

Manual Técnico

Torneo de Sudoku: Validación y Análisis de Partidas (LFP Numerix)

Lenguajes Formales y de Programación — Universidad de San Carlos de Guatemala

1. Descripción general

El sistema es un programa de consola escrito en Python 3, desarrollado con Programación Orientada a Objetos. Su función es leer archivos de texto delimitados por comas con información de sudokus, jugadores e intentos de resolución, validar cada intento contra las reglas del Sudoku mediante matrices, calcular métricas de desempeño y generar reportes en formato HTML.

2. Estructura del proyecto

El código está organizado en módulos según su responsabilidad:

```
Practica 1/
├── main.py
├── datos/
│   ├── sudokus.lfp
│   ├── jugadores.lfp
│   └── intentos.lfp
├── Modelos/
│   ├── tablero.py
│   ├── jugador.py
│   ├── intento.py
│   └── torneo.py
├── io_archivos/
│   ├── lector.py
│   └── generador_de_reportes.py
├── Logica/
│   ├── validador.py
│   └── metricas.py
├── Interfaz/
│   └── menu.py
└── reportes/
```

3. Clases y objetos

Clase	Atributos principales	Responsabilidad
Tablero	id_sudoku, dificultad, tablero_original, matriz	Representar un sudoku y convertirlo en matriz 9x9
Jugador	carnet, nombre, apellido, nivel	Representar a un jugador inscrito
Intento	jugador, tablero, solucion, matriz_solucion, tiempo_segundos, fecha, validez_porcentaje, resuelto_correctamente	Representar el intento de resolución de un jugador sobre un tablero

Torneo	sudokus, jugadores, intentos	Coordinar la carga de las tres listas de datos
--------	------------------------------	--

Cada clase se guarda en su propio archivo dentro de la carpeta Modelos. La clase Torneo no valida ni calcula nada por sí misma: solo agrupa las tres listas y delega la lectura de archivos a las funciones del módulo lector.py. Esta separación evita mezclar la responsabilidad de "guardar datos" con la de "leer archivos" o "validar reglas".

4. Lectura de archivos

El módulo io_archivos/lector.py contiene tres funciones: cargar_sudokus, cargar_jugadores y cargar_intentos. Las tres siguen el mismo patrón:

- Abren el archivo con open() dentro de un bloque with, en modo lectura ("r").
- Leen todas las líneas con readlines().
- Recorren cada línea con un for, ignorando líneas vacías.
- Separan los campos con el método split(',').
- Construyen un objeto (Tablero, Jugador o Intento) por cada línea válida.
- Están envueltas en un bloque try/except que captura FileNotFoundError y errores inesperados, para que una ruta incorrecta no detenga la ejecución del programa.

La función cargar_intentos requiere además las listas de jugadores y sudokus ya cargadas, porque un Intento no guarda el carnet o el id como texto, sino el objeto Jugador y el objeto Tablero correspondientes. Por esto, el orden de carga en el menú es obligatorio: primero sudokus y jugadores, después intentos.

5. Representación matricial

Cada tablero y cada solución propuesta se guardan como una cadena de 81 caracteres. El método generar_matriz (en Tablero) y generar_matriz_solucion (en Intento) convierten esa cadena en una lista de 9 listas de 9 elementos (matriz 9x9), recorriendo la cadena en bloques de 9 caracteres.

Pseudocódigo de la conversión:

```
matriz = []
para i desde 0 hasta 81 en pasos de 9:
    fila_texto = cadena[i : i + 9]
    fila_numeros = convertir cada caracter de fila_texto a entero
    agregar fila_numeros a matriz
```

6. Lógica de validación

El módulo Logica/validador.py contiene la lógica central del sistema. Se decidió que las funciones para extraer una fila, una columna o una caja de una matriz fueran funciones independientes (no métodos de la clase Tablero), porque se necesitan tanto para la matriz del tablero original como para la matriz de la solución propuesta por el jugador.

6.1 Verificación de pistas

La función verificar_pistas compara, celda por celda, la matriz original del tablero contra la matriz de la solución propuesta. Si una celda del tablero original tiene un valor distinto de cero (una pista) y ese valor cambió en la solución, el intento se marca como inválido en ese aspecto.

```

para cada fila de 0 a 8:
    para cada columna de 0 a 8:
        si valor_original != 0 y valor_original != valor_propuesto:
            retornar False
retornar True

```

6.2 Validación de filas, columnas y cajas

La función `es_grupo_valido` recibe una lista de 9 números y verifica que, al ordenarla, sea exactamente [1, 2, 3, 4, 5, 6, 7, 8, 9]. Esta misma función se reutiliza para validar las 9 filas, las 9 columnas y las 9 cajas de 3x3, extraídas con `obtener_fila`, `obtener_columna` y `obtener_caja`.

La extracción de una caja se calcula a partir del número de caja (0 a 8), obteniendo la fila y columna donde inicia:

```

fila_inicio = (numero_caja // 3) * 3
columna_inicio = (numero_caja % 3) * 3

```

6.3 Cálculo del porcentaje de validez

La función `validar_intento` cuenta cuántas de las 27 agrupaciones (9 filas + 9 columnas + 9 cajas) cumplen la regla, y calcula el porcentaje con la fórmula indicada en el enunciado:

```

porcentaje = (grupos_correctos / 27) * 100

```

Un intento se marca como `resuelto_correctamente = True` únicamente si el porcentaje es 100 y, además, `verificar_pistas` devuelve `True`.

7. Cálculo de métricas

El módulo `Logica/metricas.py` contiene funciones genéricas que reciben una lista de intentos y devuelven un valor calculado:

- `tiempo_promedio`: promedio de `tiempo_segundos` sobre la lista recibida.
- `validez_promedio`: promedio de `validez_porcentaje` sobre la lista recibida.
- `tasa_de_exito`: porcentaje de intentos con `resuelto_correctamente = True`.
- `intentos_por_sudoku` e `intentos_por_jugador`: filtran la lista completa de intentos por `id_sudoku` o por `carnet`.

Estas funciones no distinguen si reciben todos los intentos del torneo o un subconjunto ya filtrado, lo que permite reutilizarlas en los tres reportes sin duplicar lógica.

8. Generación de reportes HTML

El módulo `io_archivos/generador_de_reportes.py` contiene tres funciones, una por cada reporte solicitado:

- `generar_reporte_sudokus`: recorre la lista de tableros y, para cada uno, filtra sus intentos con `intentos_por_sudoku` y calcula tiempo promedio y tasa de éxito.
- `generar_reporte_jugadores`: recorre la lista de jugadores y, para cada uno, filtra sus intentos con `intentos_por_jugador` y calcula validez promedio, tiempo promedio y cantidad de tableros resueltos perfectamente.
- `generar_reporte_top10`: filtra los intentos con `resuelto_correctamente = True`, los ordena por `tiempo_segundos` con `sorted()` y toma los primeros 10.

Cada función arma el documento HTML concatenando texto en una variable, y lo guarda con `open(ruta, "w")` dentro de la carpeta reportes/.

9. Menú de consola

El módulo `Interfaz/menu.py` implementa un ciclo `while` que se repite mientras la opción elegida no sea "8". Cada opción del menú llama a un método de Torneo o a una función de validación, métricas o reportes. Antes de generar un reporte o validar, se verifica que la lista correspondiente no esté vacía, mostrando un mensaje claro en caso contrario.

10. Manejo de excepciones

Las tres funciones de carga en `lector.py` están envueltas en un bloque `try/except` que captura específicamente `FileNotFoundError` (ruta de archivo incorrecta) y, de forma general, `Exception` para cualquier otro error no previsto. En ambos casos se imprime un mensaje descriptivo y la función devuelve una lista vacía, evitando que el programa se detenga de forma abrupta.